

ArchSync: Evidence-Backed Conformance Checking for TypeScript Service Architectures

ANONYMOUS AUTHOR(S)

Service changes can introduce architectural drift while preserving functional behavior, leaving maintainers without a clear link between a violated design rule and the code responsible. ArchSync combines an explicit architectural contract, provenance-aware extraction of selected TypeScript service interactions, and a Git gate that distinguishes introduced violations from changes requiring an architecture decision. We investigate implementation correctness using co-developed patches and detector signals, followed by finite rule checking and adversarial testing. All 20 development patches match their declared classifications, but targeted adversarial inputs expose extraction and incremental-analysis defects that require repair. Retained failing inputs and repaired outputs make these interventions inspectable. The contribution is a reproducible integration of established conformance concepts with service-level evidence and change-time decisions. The results establish bounded development behavior, not real-world accuracy or superiority over existing tools; independent repository evaluation and an executed external comparison remain necessary.

CCS Concepts: • **Software and its engineering** → **Software architecture**; *Software verification*.

Additional Key Words and Phrases: architecture conformance, service-oriented systems, static analysis, architecture drift, reproducibility

ACM Reference Format:

Anonymous Author(s). 2027. ArchSync: Evidence-Backed Conformance Checking for TypeScript Service Architectures. In *Draft for ICIT 2027 (Ho Chi Minh City)*. ACM, New York, NY, USA, 10 pages.

1 INTRODUCTION

Service-oriented systems can preserve functional behavior while introducing a forbidden dependency, losing a required interaction, or adding infrastructure that needs a design decision. These changes concern different architectural judgments. A useful check must connect an observed interaction to the declared contract, identify supporting source evidence, and distinguish a violation from an evolution. ArchSync addresses this information-extraction and decision-support problem with deterministic rules and inspectable evidence.

Our contribution is an executable integration: (1) a typed expected/observed service graph with explicit constraints; (2) provenance-aware extraction of selected TypeScript interactions and evidence; and (3) a Git-diff gate that reports introduced violations separately from evolution. Expected/observed comparison and dependency rules are established concepts, not new algorithms introduced here. The contribution examined is their integration into an inspectable service-level change gate and a failure-first account of its implementation limits. We do not establish that this combination is unique, more accurate than existing tools, or more useful to developers.

We ask whether the prototype reconstructs declared graph changes and detector signals (RQ1), classifies controlled changes correctly (RQ2), supplies the declared evidence locations (RQ3), and preserves results under replay and incremental execution (RQ4). These questions guide the controlled evaluation in Section 4 and the reliability extension in Section 5. For practitioners, the Git-scoped gate provides a concrete pull-request check: introduced findings carry rule

A note.

2027. Manuscript submitted to ACM

Table 1. Published context, not an executed baseline comparison. Different languages, units and datasets prevent numerical ranking.

Approach	Unit and decision	Evaluation evidence
ArchLintor [13]	TypeScript module dependencies; allowed, forbidden, required	One author-developed eight-module example; inserted situations
CALint [3]	Python dependencies; Clean Architecture rule	Three cases reported in abstract; full text unavailable
Recovery comparison [14]	Components, connections, endpoints; recovery correctness	Nine tools on 17 Java/Spring applications
ArchSync (this work)	Typed service relations; rules and introduced findings	20 co-developed patches; 40 detector signals; no independent holdout

identifiers and inspectable evidence, while evolution is routed for an explicit architecture decision. The decision path executes deterministic rules without an LLM inference service.

2 BACKGROUND AND RELATED WORK

This scoped narrative review positions ArchSync against foundational and contemporary conformance approaches. Sources were selected purposively from the retained keyword-search corpus, not through a completed systematic review. Contemporary evidence is from 2022–2025; two older references identify the origins of model comparison and dependency constraints, not current tool performance.

Problem and observation boundary. A recent mapping characterizes architecture erosion [8]. Code-review studies identify symptoms in OpenStack and OpenStack/Qt projects [7, 9]; their project overlap limits independence. Practitioner evidence identifies documentation and shared practices as ways to manage drift [1]. A recovery taxonomy and microservice-tool mapping distinguish reconstruction purposes and applicability [2, 12]. View-based drift analysis compares intended and reconstructed specifications in an industrial case [16]. These works motivate an explicit observation boundary rather than equating a partial source graph with the complete runtime architecture.

Closest conformance approaches. Software Reflexion Models establish high-level/source model comparison [11], and Dependency Constraint Language develops declarative dependency constraints [15]. ArchSync inherits these concepts. CALint checks Python dependencies against Clean Architecture rules [3]. ArchLintor already detects TypeScript module-interaction violations, provides source-linked diagnostics and supports CI/CD integration [13]. Thus neither TypeScript checking nor a CI gate alone is a novelty claim. LLM-generated architectural tests introduce a different translation step, where executing a test does not establish correct expression of the intended rule [10]; ArchSync’s runtime decisions instead use explicit deterministic rules.

Change-time feedback and verification. Repository mining studies microservice evolution [4], and industrial conformance visualization reports practitioner experience [17]. ArchSync joins typed service observations to introduced-finding keys at a Git head. Evaluation-method research stresses replicability [6]; interrogation testing shows that targeted inputs can expose analyzer soundness and precision failures [5]. These motivate retained failure/repair evidence, not a claim that passing constructed tests estimates field accuracy.

Comparative boundary. Table 1 summarizes publications, not tools rerun on our data. ArchLintor’s inserted absence/divergence examples do not supply a precision/recall benchmark. CALint’s publisher abstract reports three cases; unavailable full text prevents a finer audit. Schneider et al. execute nine recovery tools on 17 Java/Spring applications [14], illustrating an external comparison design rather than an ArchSync performance baseline. Our integration targets HTTP, PostgreSQL, Redis and AMQP service relations with evidence and introduced-finding decisions. We have

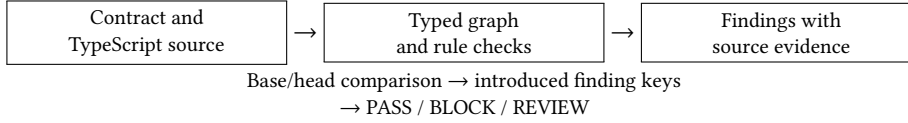


Fig. 1. Source observations and explicit contracts feed a change-time decision. The model remains under separate human control.

not established a unique capability advantage or measured superiority over these approaches. Section 6 defines the remaining comparative requirement.

3 EXECUTABLE CONTRACT AND CHANGE-TIME ANALYSIS

Let $M = (V, E, R)$ be the expected architecture, with components V , directed typed relationships E , and rules R . The observed graph O is reconstructed from supported source constructs. Component identifiers and metadata from M guide reconstruction. Edges collapse repeated observations with the same source, target and kind, while retaining their file/line evidence.

A deny rule rejects matching edges. An allow rule restricts destinations of the selected outgoing relations. A require rule demands a matching direct edge for each selected source; require-path demands a nonempty directed path, optionally restricted to one edge type. An observed rule violation takes classification precedence; otherwise a graph difference is evolution, and an unchanged graph is no-impact. The expected model remains under explicit maintainer control.

The TypeScript adapter recognizes selected global fetch calls and imported PostgreSQL, Redis and AMQP client operations. Import provenance distinguishes package clients from method-name lookalikes. HTTP targets use host identity; AMQP edges capture broker reachability. Evidence scores are heuristic. Section 6 describes the resulting observation boundary.

For a base/head pair, the gate reconstructs the base, analyzes affected source components and merges retained observations with new ones. Full-head scanning supplies a consistency oracle. New violation keys produce BLOCK, new evolution keys produce REVIEW, and no introduced finding produces PASS. Existing findings remain visible. Keys operate at relationship level, so an additional offending call on an existing edge need not introduce a new key. PASS denotes the absence of newly recognized gated findings. REVIEW requires a separate architecture decision; unchanged inputs remain REVIEW after a human rejection.

The expected model supplies the input contract. Call-site evidence points to observed code. A missing required relation has no absent call to localize, so it receives a deterministic component anchor. These evidence types are scored separately. Cache identity binds the baseline and configuration; later hardening also separates repository scope and analyzer version and checks payload integrity. A checksum detects accidental payload corruption.

Implementation artifacts. Core implements model validation, graph comparison and rules; Guardian implements source extraction and the Git gate. Historical measurements use pinned Core and Guardian revisions supplied in the review artifact. The later local hardening candidate is separately identified by the packaged source archive and SHA-256 manifest, not substituted for historical revisions.

Worked controlled example. In D1 case 06, rule ARCH-001 forbids a direct relationship from frontend to payment-service. The patch adds `payDirectly` and a fetch call to the payment endpoint. Reconstruction adds one HTTP edge between those components; the deny rule produces an error finding with evidence at `frontend/src/app.ts`, line 14 of the patched file. Because the finding is introduced relative to the base, the Git gate

Table 2. Agreement with co-developed expectations on one synthetic system. These are implementation checks, not independent accuracy estimates.

Measure	Agreement/count
D1 changed nodes / edges	5/5; 12/12
D1 classifications / violation rule sets	20/20; 7/7
Call-site cases / missing-relation anchors	9/9; 2/2
P3 decisions / exact deltas	20/20; 20/20
P3 incremental/full scan / cold–warm replay	20/20; 20/20
P3 parsed / head file instances	57/189

returns BLOCK. A new relation without a violated rule instead produces REVIEW, reserving approval for a separate architecture decision.

4 CONTROLLED IMPLEMENTATION VERIFICATION

Objects and independence. D1 is a synthetic Order Platform baseline with five components and five relationships. Its 20 isolated patches comprise nine no-impact changes, seven violations and four evolutions. Topology labels preceded integration, but executable patches and precise evidence locations co-evolved with the analyzer. D2 has four positive and four hard-negative signals for each of five detector groups: fetch, PostgreSQL, Redis, AMQP publish and AMQP consume. It was constructed from v0.1 weaknesses and informed v0.2. P3 replays the same D1 patches through Git.

Procedure and units. D1 validates hashes, patch applicability and expected changes, then runs the baseline and each isolated patch twice (42 executions). D2 runs both pinned versions twice (four executions). P3 creates fresh base-line repositories, applies one patch, performs cold and warm checks, and separately scans the full head. Results are compared after excluding elapsed time and cache-hit state. The 20 cold/warm pairs plus full scans require 80 analyzer calls. D1 items are graph nodes/edges; D2 items are annotated detector–edge–file–line signals, which can share graph edges. Evidence matches use the declared case-level locations.

Results. Table 2 reports the meaningful changed-item and case denominators. Full-graph totals also matched 105 nodes and 108 edges, but repeatedly include baseline structure. On D2, v0.1 yielded TP=18, FP=4, FN=2 and TN=16; v0.2 yielded TP=20, FP=0, FN=0 and TN=20. The four false positives were local PostgreSQL/AMQP-shaped receivers; the two misses were Redis operations. These targeted repairs improve the within-tool regression results. Duplicate normalized output matched, and P3 cache miss/hit results agreed in all 20 cases.

The original Windows recording used Node.js 22.16.0 and an Intel Core i7-13650HX. Cold/warm nearest-rank p50 values were 518.51/242.62 ms and p95 values 531.05/249.30 ms. They combine parsing, Git, cache I/O and reporting; RAM, power/load and filesystem-cache state were unrecorded. Historical functional replays passed Linux and Windows; this checks installation and repeatability on the same inputs, not independent accuracy, representative workloads or comparative speed. SHA-256 identifies retained bytes but cannot establish that their labels are correct.

Data access and interpretation. The review artifact identifies the pinned benchmark revision and input hashes. D1/P3 share one system and the same patches; D2 was used to improve the detector. Perfect matches are therefore regression-oracle agreement, not success on an untouched holdout. The v0.1/v0.2 comparison is within-tool development evidence, not an external baseline.

5 SEPARATE TECHNICAL RELIABILITY EXTENSION

The subsequent 2026-09-26 campaign targets local development defects and preserves original failures. The preceding observations retain their original pinned analyzer packages.

Method provenance. OpenAI Codex with delegated coding agents assisted in constructing the finite oracle, adversarial inputs and mutation harness, modifying the implementation, executing local checks and inspecting outputs. Reported counts come from retained executable test reports. The development process includes automated cross-agent code review and test refinement after failure inspection, as detailed below.

Finite rule checking. A separately implemented adjacency-matrix and transitive-closure oracle checks all 4,096 directed graphs on three fixed nodes, two edge types and 12 non-self edge slots. Each graph is evaluated against 192 combinations of deny/allow/require/require-path, exact/wildcard selectors and type constraints: 786,432 rule-graph evaluations. Additional checks apply ordering and consistent renaming on 241 graph states and inverse addition/removal deltas on all 4,096.

Test sensitivity. Eight declared implementation mutations target rule enforcement, path traversal and classification. The initial suite killed seven; violation demotion survived because classification was not asserted. Adding the missing assertion produced eight kills, no survivors and no execution errors in the final audit. Both rounds are retained. The two rounds document increased sensitivity to the declared fault model.

Extraction and incremental faults. Adversarial source tests exposed type-only imports treated as runtime clients, protocol mismatches, shadowed bindings and mistaken URL expressions. Binding symbols per file separates local names from package/global names. Later checks exposed wrong-host string concatenation and stale provenance after direct reassignment. Static string composition now resolves the complete URL. A further 14-case extension reproduced 11 failures caused by destructuring, unary updates and loop writes; all 43 cumulative adversarial cases pass after repair. Recognized writes conservatively invalidate the binding throughout its file, including earlier legitimate uses. Unresolved composition emits no relationship.

Incremental checks exposed missed unusual/ignored filenames, nested-component evidence loss, cross-project cache reuse and corrupt-cache behavior. Exact source ownership, Git path handling, repository/version-bound cache identity and payload checks address the reproduced cases. The extension records failing inputs, repaired outputs and remaining limitations separately. Its full local suites passed 126 Core and 315 Guardian tests on Node.js 22.16.0/macOS arm64. Two inert compiler-host callbacks are excluded from Guardian’s configured coverage. The current analyzer is unreleased semantics version 0.4; it also passes a development replay of all 20 D1 cases.

6 VALIDITY AND IMPLICATIONS

Construct and internal validity. D1 and D2 are controlled, co-developed datasets; P3 reuses D1. Repeated baseline structure and replay executions therefore provide consistency checks rather than independent observations. Shared assumptions between labels, implementation and the separately coded oracle can survive these checks. D2 covers declared detector signals and hard negatives, and file/line agreement measures specified case locations rather than complete evidence recall. The typed structural view addresses conformance, leaving responsibilities, runtime quality and practitioner usefulness for separate evaluation.

Analysis and decision boundary. Reconstruction depends on the supplied model and supported source constructs. HTTP host identity collapses port/path distinctions, and AMQP broker reachability leaves routing compatibility unresolved. Dynamic templates, alternative fallbacks, property mutation, imported-constructor rebinding, wrappers and

inter-file flows can remain invisible. File-wide binding invalidation can suppress legitimate earlier uses; the extractor is neither sound nor flow-sensitive. Evidence scores are uncalibrated heuristics. Relationship-level keys can hide an additional offending call on an existing edge, and incremental/full agreement can preserve a shared omission. PASS is limited to newly recognized findings; REVIEW leaves approval and rejection enforcement to repository governance. Cache checksums address accidental corruption, not a writer able to replace both payload and checksum.

Reliability scope. The oracle enumerates a finite three-node domain, excluding self-loops, larger graphs, arbitrary wildcard fragments, extraction and component-metadata changes. Its 786,432 combinations are not independent samples. Mutation testing covers eight declared faults and includes a post-failure assertion repair; it is a test-development result rather than a whole-program mutation score. Regression counts describe executed checks. Automated cross-agent review supplies no independent human annotation, adjudication or validation signoff. Historical Linux/Windows replay and the later macOS hardening campaign have distinct execution scopes.

External and comparative validity. The controlled results do not establish representative TypeScript accuracy, comparative superiority or developer productivity. An independently labeled real-repository holdout and an executed external-tool comparison remain future work. Table 1 summarizes published context: recovery-characteristic scores and ArchSync patch agreement use different units and cannot be ranked. A fair comparison requires a nonempty common relation vocabulary, frozen inputs and explicit reporting of unsupported cases. Module-import tools such as dependency-cruiser cannot be scored against HTTP or database edges by treating unsupported relations as misses. A matched adapter must be validated and frozen before holdout selection, or a compatible comparator must be selected. The purposive narrative review is not exhaustive; missing full texts remain recorded, and overlapping project contexts in the erosion studies limit their independence.

Conclusion validity and next steps. Small changed-item counts and finite enumeration do not justify population significance claims. Recorded timings combine parsing, Git, cache I/O and reporting, with RAM, system load and filesystem-cache state unrecorded; causal speedup and scalability require a paired controlled study. Broader evaluation should freeze projects and tools before predictions, label the full declared source scope, retain disagreements and report per-project errors, coverage and failures. Practitioner evaluation should measure false-block burden and review effort.

7 CONCLUSION

ArchSync integrates typed service-level observations, explicit conformance rules and evidence-backed Git decisions. The controlled evaluation demonstrates agreement across 20 development patches and incremental/full replay; the reliability campaign exercises the finite rule contract and repairs reproducible extraction and cache defects. Together, these artifacts provide a reproducible foundation for architecture checks at pull-request time. Maintainers can inspect the evidence behind introduced findings and distinguish rule violations from changes requiring an architecture decision. Future work extends evaluation to independently labeled repositories, compatible external baselines and practitioner workflows.

REFERENCES

- [1] Emilie Anthony, Astrid Berntsson, Tiziano Santilli, and Rebekka Wohlrab. 2024. We’re Drifting Apart: Architectural Drift from the Developers’ Perspective. In *2024 IEEE 21st International Conference on Software Architecture*. IEEE, Hyderabad, India, 101–111. <https://doi.org/10.1109/ICSA59870.2024.00018>
- [2] Alexander Bakhtin, Xiaozhou Li, Jacopo Soldani, Antonio Brogi, Tomas Cerny, and Davide Taibi. 2024. *Tools Reconstructing Microservice Architecture: A Systematic Mapping Study*. Springer Nature Switzerland, 3–18. https://doi.org/10.1007/978-3-031-66326-0_1

- [3] Clevio Orlando de Oliveira Junior, Jonathan Carvalho, Fábio Fagundes Silveira, Tiago Silva da Silva, and Eduardo Martins Guerra. 2022. CALint: A Tool for Enforcing the Clean Architecture's Dependency Rule in Python. In *Computational Science and Its Applications – ICCSA 2022 Workshops: Malaga, Spain, July 4–7, 2022, Proceedings, Part V*. Springer International Publishing, 534–549. https://doi.org/10.1007/978-3-031-10548-7_39
- [4] Patric Genfer and Uwe Zdun. 2024. Exploring Architectural Evolution in Microservice Systems Using Repository Mining Techniques and Static Code Analysis. In *Software Architecture: 18th European Conference, ECSA 2024, Luxembourg City, Luxembourg, September 3–6, 2024, Proceedings*. Springer Nature Switzerland, 157–173. https://doi.org/10.1007/978-3-031-70797-1_10
- [5] David Kaindlstorfer, Anastasia Isychev, Valentin Wüstholtz, and Maria Christakis. 2024. Interrogation Testing of Program Analyzers for Soundness and Precision Issues. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ACM, New York, NY, USA, 319–330. <https://doi.org/10.1145/3691620.3695034>
- [6] Marco Konersmann, Angelika Kaplan, Thomas Kühn, Robert Heinrich, Anne Kozirolek, Ralf H. Reussner, Jan Jürjens, Mahmood al Doori, Nicolas Boltz, Marco Ehl, Dominik Fuchß, Katharina Großer, Sebastian Hahner, Jan Keim, Matthias Lohr, Timur Sağlam, Sophie Schulz, and Jan-Philipp Töberg. 2022. Evaluation Methods and Replicability of Software Architecture Research Objects. In *2022 IEEE 19th International Conference on Software Architecture (ICSA)*. IEEE, Honolulu, HI, USA, 157–168. <https://doi.org/10.1109/ICSA53651.2022.00023>
- [7] Ruiyin Li, Peng Liang, and Paris Avgeriou. 2023. Warnings: Violation symptoms indicating architecture erosion. *Information and Software Technology* 164 (2023), 107319. <https://doi.org/10.1016/j.infsof.2023.107319>
- [8] Ruiyin Li, Peng Liang, Mohamed Soliman, and Paris Avgeriou. 2022. Understanding Software Architecture Erosion: A Systematic Mapping Study. *Journal of Software: Evolution and Process* 34, 3 (2022), e2423. <https://doi.org/10.1002/smr.2423>
- [9] Ruiyin Li, Mohamed Soliman, Peng Liang, and Paris Avgeriou. 2022. Symptoms of Architecture Erosion in Code Reviews: A Study of Two OpenStack Projects. In *2022 IEEE 19th International Conference on Software Architecture (ICSA)*. IEEE, 24–35. <https://doi.org/10.1109/icsa53651.2022.00011>
- [10] Andrielly Lucena, Everton L. G. Alves, and João Brunet. 2025. Language Models as Architectural Gatekeepers: Automating Conformance Checking from Natural Language. In *Anais do XXXIX Simpósio Brasileiro de Engenharia de Software (SBES 2025)*. Sociedade Brasileira de Computação, 713–719. <https://doi.org/10.5753/sbes.2025.11100>
- [11] Gail C. Murphy, David Notkin, and Kevin J. Sullivan. 1995. Software Reflexion Models: Bridging the Gap Between Source and High-Level Models. In *Proceedings of the Third ACM SIGSOFT Symposium on Foundations of Software Engineering*. ACM, New York, NY, USA, 18–28. <https://doi.org/10.1145/222124.222136>
- [12] Abdul Qayum, Mengqi Zhang, Simon Colreavy, Muslim Chochlov, Jim Buckley, Dayi Lin, and Ashish Rajendra Sai. 2024. A Framework and Taxonomy for Characterizing the Applicability of Software Architecture Recovery Approaches: A Tertiary-Mapping Study. *Software: Practice and Experience* (2024). <https://doi.org/10.1002/spe.3364> First published online 17 July 2024.
- [13] Matheus Resende, Rafael Durelli, and Ricardo Terra. 2025. ArchLintor: detecting architectural violations in TypeScript systems. In *Anais do XXXIX Simpósio Brasileiro de Engenharia de Software (SBES 2025)*. Sociedade Brasileira de Computação, 900–906. <https://doi.org/10.5753/sbes.2025.11075>
- [14] Simon Schneider, Alexander Bakhtin, Xiaozhou Li, Jacopo Soldani, Antonio Brogi, Tomas Cerny, Riccardo Scandariato, and Davide Taibi. 2025. Comparison of static analysis architecture recovery tools for microservice applications. *Empirical Software Engineering* 30, 5 (June 2025). <https://doi.org/10.1007/s10664-025-10686-2>
- [15] Ricardo Terra and Marco Tulio Valente. 2009. A Dependency Constraint Language to Manage Object-Oriented Software Architectures. *Software: Practice and Experience* 39, 12 (2009), 1073–1094. <https://doi.org/10.1002/spe.931>
- [16] Burak Uzun and Bedir Tekinerdogan. 2024. Detecting Deviations in the Code Using Architecture View-Based Drift Analysis. *Computer Standards & Interfaces* 87 (2024), 103774. <https://doi.org/10.1016/j.csi.2023.103774>
- [17] Filip Zamfirov, Andrei Radulescu, Jacob Krüger, and Michel R. V. Chaudron. 2025. Lessons from Visualizing Software Architecture Structure Conformance at Thermo Fisher Scientific. In *Software Engineering and Advanced Applications: 51st Euromicro Conference, SEAA 2025, Salerno, Italy, September 10–12, 2025, Proceedings, Part III*. Springer Nature Switzerland, 372–389. https://doi.org/10.1007/978-3-032-04207-1_25

A REPRODUCTION AND INTERPRETATION DETAILS

This appendix supplies operational detail for the single-column review profile. It is supplied as a separate supplement with the compact profile; acceptance of supplementary uploads is not established by the public instructions. Its purpose is to make the development evidence inspectable, not to add independent observations.

A.1 Keep historical and current evidence separate

Historical D1/D2/P3 observations are reproduced from their pinned benchmark inputs and packaged analyzers. The reporting derivation uses benchmark commit 24d63ebf2fc3075a1d64f1eaff38cdc0b7f586fb. Its ground truth, Phase

2 and Phase 3 records have individual hashes. Rerunning a current analyzer on that same corpus answers a different question: whether the updated implementation preserves known development behavior. The latter cannot silently replace the former in a before/after table.

The artifact therefore distinguishes original packages, current source snapshots, current distribution builds, first failing runs, repaired runs and unchanged historical result inputs. Current source and package hashes identify the evaluated candidate; an unreleased version string alone does not identify its bytes. The accompanying manifest maps each new manuscript count to its raw receipt. A successful package installation is an operational compatibility check, not an independent assessment of architectural correctness.

The historical evidence-location decomposition is explicitly post-hoc. Nine finding-bearing cases concern call sites; cases 08 and 17 concern deterministic anchors for missing edge/path requirements. Case 15 contains two violated rules but counts once. A match in these finite cases cannot establish that all useful evidence was found, correctly ranked or understood by a developer.

A.2 Finite oracle construction

For three nodes, six ordered non-self node pairs and two edge types give 12 binary edge slots, hence $2^{12} = 4096$ graphs. For each graph, an adjacency matrix is built independently for all edges and for each individual edge type. Direct requirements query that matrix; path requirements query its transitive closure. The diagonal is not initially true: a required path must contain an edge. A cycle may nevertheless establish a nonempty path back to its starting node.

For each of four rule kinds, the source and target independently range over three exact identifiers and a full wildcard, and the edge-type restriction ranges over unrestricted and the two types. Thus $4 \times 4 \times 4 \times 3 = 192$ rule configurations are checked for every graph. The oracle compares per-rule violation counts, aggregate violations and classification. It imports no production graph-building, selector-matching or traversal helper. This structural separation reduces direct implementation reuse but does not establish independent authorship or correctness of the shared rule interpretation.

Ordering and consistent-renaming checks use masks 0 through 4080 in steps of 17, giving 241 graph states. Inverse-delta checks compare every graph with the empty-edge graph. They verify that added edges in one direction are removed edges in the other, and that edge changes without rules classify as evolution. These checks do not enumerate arbitrary pairs of graphs, metadata changes, arbitrary wildcard fragments or source-language programs. The chosen domain is exhaustive only within its declared bounds.

A.3 Mutation procedure and retained failures

The mutation harness copies the relevant source and test into a temporary isolated workspace, changes one declared expression at a time, and invokes the same test suite with a fixed timeout. Production source remains unchanged by this procedure. Every run retains the replacement text, original and mutated source hashes, test hash, command output, exit state and native report. A passing unmodified baseline is required before mutant outcomes are interpreted.

Mutant states distinguish test passes, assertion failures and execution errors. The eight faults disable deny enforcement, invert allow logic, invert direct-require satisfaction, ignore path edge types, prevent traversal beyond one edge, miss path target detection, demote violation, or demote evolution. The scope is intentionally narrow. A compile error or timeout is not silently counted as an assertion kill, and no claim about all equivalent mutants is made.

The original surviving classification mutation matters: it demonstrated that correct finding counts alone did not guarantee the test checked the returned decision. After the assertion was added, the same fault was detected. Reporting

only the final perfect score would hide that development intervention. Both versions of the test evidence remain available, so reviewers can distinguish test strengthening from confirmation on an untouched sample.

A.4 Detector repairs and residual uncertainty

The first adversarial set contains 18 cases: 15 failed before intervention and all 18 passed after it. Positive controls check aliases, captured resources and an unshadowed global fetch alongside local bindings. Type-only import tests deliberately include source that a TypeScript compilation gate would reject. They test the analyzer’s robustness when parsing source without that gate, not the prevalence of such constructs in real projects.

A later intervention added eight cases for concatenation and reassignment: seven failed in the cumulative 26-case pre-repair suite. Three further boundary controls were added after repair, yielding 29 passing adversarial cases. These before/after suites therefore have different total sizes, and a naive percentage comparison would be misleading. An old positive fixture concatenated a suffix into an invalid URL port; the fixture was corrected to a valid path suffix and the invalid-port input retained as a negative regression.

A third intervention adds 14 cases to the 29-case suite: 12 write scenarios and two controls for unrelated or shadowed bindings. The identical 43-case suite has 11 failures and 32 passes before the change, then 43 passes afterward. One added write scenario already passed. The repair resolves variable identities within array/object destructuring (including rest/default targets), increment/decrement operations and assignment targets in for-in/for-of loops. For example, a URL binding overwritten by an array assignment must not retain its initializer as the target of a later fetch. These constructed programs test specified behavior; they are not independent observations of real-world prevalence.

Recognized-write invalidation is conservative and file-wide. It can suppress a genuine call that appears before re-assignment; it does not implement statement-order data flow. Property mutation, imported constructor rebinding, dynamically built templates and alternative fallback destinations remain separate risks. When no edge is emitted, the tool does not prove that no runtime relationship exists. Confidence constants attached to emitted evidence cannot resolve this missing-observation uncertainty. Analyzer version 0.4 invalidates caches from earlier semantics. The full repository gate passed from a clean temporary candidate whose 410 tracked files matched the working source byte-for-byte. The first full run stopped at a CLI assertion still expecting version 0.3; that failure is retained alongside the corrected assertion and passing rerun. These are packaging and regression checks, not independent reproduction.

A.5 Incremental and cache experiment boundary

The incremental suite creates temporary Git repositories and checks additions, modifications, deletions, renames within and between components, and complete-component deletion. Cold and warm results are compared with fresh full scans. The initial nine adversarial cases all failed on the original implementation; a later ignored-source case exposed a further false PASS. Before and after receipts preserve those observations.

Most differential cases compare classification, decisions, finding count and scanned-file count. Cold/warm comparisons additionally inspect architecture deltas and introduced/resolved findings. The nested-component case compares the complete observed graph and evidence. This distinction is important: aggregate count agreement alone does not prove identity of every graph edge. The suite does not constitute a proof over all Git configurations or filesystem layouts.

The baseline cache now separates project-relative roots, architecture identity and analyzer semantics. A payload hash rejects accidentally changed JSON; concurrent writers use distinct temporary publication paths. Neither mechanism authenticates a malicious cache writer. Git inventory and scanning are not an atomic snapshot, so the worktree

must remain stable during a check. Ignored source outside excluded build/dependency directories is included because full scanning would read it; the gate must not ignore a relationship merely because Git does.

A.6 Independent follow-up design

The technical extension specifies a future repository frame selected before prediction inspection, with lineage-aware development/pilot/holdout separation. Two independent human reviewers must enumerate supported relations across the complete declared source scope, seal their separate labels, and preserve disagreements before adjudication. Checking emitted predictions alone can estimate precision but cannot establish recall. Supported-vocabulary coverage, execution failures and unresolved labels require separate accounting.

A comparator must share a nonempty, frozen observation unit with ArchSync. An import dependency and a remote service interaction are not interchangeable merely because both can be drawn as graph edges. Performance experiments must pair the same workloads and machines, control the graph-cache condition, separate priming from timed execution and retain failures. Operating-system disk cache is not equivalent to the ArchSync graph cache. These protocols are proposed technical designs, not completed studies or evidence of independent human participation.